Master Protocol for Invariant-Based Security Co-Architecture with AI Agents

A Symbiotic Architecture Protocol for Structural Security Auditing

Gonzalo Emir Durante
Independent Researcher
duranteg2@gmail.com

August 13, 2026

Abstract

The escalating complexity of software and artificial intelligence ecosystems has exposed a fundamental limitation in contemporary security auditing: the inability to reason about architectural integrity beyond the surface of implementation artifacts. This paper introduces the **Symbiotic Architecture Protocol** (SAP), an operational framework for security co-architecture that validates **structural invariants** rather than enumerating isolated vulnerabilities. Unlike conventional scanners, static analyzers, or penetration testing methodologies, SAP treats security as an emergent property of architectural coherence. The protocol is materialized as an *operational specification* that orchestrates an AI agent as a **Security Co-Architect** in symbiotic partnership with a human engineer (the Origin Node).

We formalize the concept of a structural invariant as a predicate over system states that must hold under all valid execution traces. We define *invariant rupture* as the violation of such predicates, distinguishing it from traditional bug classification. The protocol integrates forensic traceability through SHA-256 cryptographic hashing and OpenTimestamps attestation, establishing an auditable chain of custody for all findings and remediation proposals.

The contribution of this work is threefold: (1) a formal methodology for invariant-based security analysis operable by large language models; (2) a six-phase operational protocol with explicit epistemic synchronization between human and machine; and (3) a reproducible, auditable framework for structural security assessment that preserves scientific rigor, legal defensibility, and strategic neutrality.

Keywords: Structural Security, Invariant-Based Auditing, Symbiotic AI Architecture, Prompt Engineering, Forensic Traceability, Security Co-Architecture.

1 Introduction

Modern security auditing is confronting an abstraction crisis. Contemporary tools—static analyzers, dynamic scanners, dependency checkers, and fuzzers—excel at detecting implementation-level anomalies: SQL injections, buffer overflows, misconfigurations, and outdated libraries [1][2]. Yet these instruments operate at the granularity of *symptoms*, not *structures*. They identify where a system fails, but rarely illuminate *why* the failure was architecturally inevitable.

Consider the canonical example: an unauthenticated user accessing another user's data. Traditional tooling might flag the specific endpoint, the missing authorization check, or the malformed JWT validation. But the *structural* failure—the rupture of the invariant "authentication gates must precede all data access decisions"—remains implicit, buried beneath layers of implementation detail. When the patch is applied locally, the invariant may still be violated elsewhere, by another route, under slightly different preconditions.

The **Security Co-Architect** model proposes a paradigm shift from reactive vulnerability enumeration to proactive invariant preservation. This model is not a replacement for penetration testing, formal verification, or compliance auditing. It is a complementary layer that operates at the level of *architectural principles*, asking not "what is broken?" but "what must never break, and where could it break?"

This paper formalizes the **Symbiotic Architecture Protocol** (SAP), originally conceived as an operational specification for large language model agents [8]. The protocol treats the AI agent not as an oracle, scanner, or substitute engineer, but as a *co-architect*—a symbiotic cognitive extension of the human engineer, bound by explicit epistemic rules, forensic documentation standards, and invariant-centered reasoning.

1.1 Contributions

The primary contributions of this work are:

1. **Formal Definition of Structural Invariants:** We introduce a mathematical framework for defining, classifying, and verifying structural invariants as predicates over system execution traces (Section 3).
2. **Symbiotic Architecture Protocol (SAP):** We present a six-phase operational protocol that structures the interaction between human engineer and AI agent, ensuring epistemic synchronization, invariant validation, and forensic traceability (Section 4).
3. **Failure Taxonomy Beyond CVSS:** We propose a classification of security failures into five structural categories—design, implementation, configuration, process, and dependency failures—decoupled from severity scoring (Section 4.4).
4. **Forensic Traceability Framework:** We demonstrate how SHA-256 hashing and OpenTimestamps attestation establish a cryptographically verifiable chain of custody for all protocol outputs, ensuring legal and scientific defensibility (Section 5).

2 Related Work

2.1 From Vulnerability Scanning to Architectural Analysis

The evolution of security auditing has progressed through three distinct paradigms. The *first wave*, spanning the 1990s to early 2000s, was dominated by signature-based scanners and rule-based static analysis tools. These systems excelled at detecting known vulnerability patterns but suffered from high false-positive rates and an inability to reason about unknown attack vectors.

The *second wave*, emerging in the 2010s, introduced model checking, symbolic execution, and fuzzing. These techniques expanded the search space of detectable flaws but remained fundamentally implementation-centric. They answer the question: "Does this code contain a path to failure?" They do not ask: "Does this architecture guarantee that such a path cannot exist?"

The *third wave*, currently nascent, involves the application of large language models to security tasks [6]. Recent work has demonstrated LLM-assisted vulnerability detection, exploit generation, and patch suggestion. However, these approaches typically treat the LLM as a *tool*—an oracle that receives code and emits findings—rather than as a *co-reasoner* engaged in architectural co-construction.

2.2 Invariants in Software Engineering

The concept of invariants has deep roots in formal methods. Hoare logic [3] introduced the notion of loop invariants as predicates preserved across iterations. Design-by-Contract [4] extended

invariants to object-oriented systems, requiring class invariants to hold before and after every public method invocation. Temporal logic enabled the expression of invariants over time, forming the basis of model checking.

However, these formalisms operate at the code or specification level. The structural invariants proposed in this work operate at the *architectural* level—predicates over the entire system topology, not merely individual functions or classes. They concern properties such as "all data flows crossing a trust boundary must be encrypted" or "no single component possesses both read and write access to the audit log."

2.3 Human-AI Symbiosis in Engineering

The notion of human-AI symbiosis traces back to Licklider's seminal 1960 essay on man-computer symbiosis [5]. In security contexts, recent work has explored collaborative red-teaming, AI-assisted threat modeling, and LLM-augmented incident response. The SAP protocol advances this lineage by formalizing the *epistemic contract* between human and machine: explicit synchronization points, shared model construction, and joint invariant design.

2.4 Threat Modeling with Large Language Models

The application of LLMs to threat modeling is an emerging research frontier. Recent experiments have demonstrated that LLMs can generate threat models from natural language system descriptions, though with significant consistency and coverage limitations. SAP differs from these approaches in three respects: (1) it operates on *invariants* rather than threat catalogs; (2) it requires empirical confirmation of every hypothesized rupture; and (3) it embeds forensic traceability as a first-class requirement rather than an afterthought.

3 Methodological Framework

3.1 Formal Definition of Structural Invariant

Let $\mathcal{S}$ be a software system comprising a set of components $C = \{c_1, c_2, \ldots, c_n\}$, a set of interfaces $I = \{i_1, i_2, \ldots, i_m\}$, and a set of execution traces $T = \{\tau_1, \tau_2, \ldots\}$, where each trace τ is a finite sequence of state transitions.

Definition 3.1 (Structural Invariant)**.** A **structural invariant** Φ is a predicate over the state space of $\mathcal{S}$ such that:

$$\forall \tau \in T, \forall s \in \tau : \Phi(s) = \text{true} \tag{1}$$

where s denotes a system state reachable during trace τ. An invariant is *structural* if its violation implies a failure of a fundamental system property (confidentiality, integrity, availability, atomicity, consistency, isolation, durability, non-repudiation).

Definition 3.2 (Invariant Rupture)**.** An **invariant rupture** occurs at state s_k in trace τ if:

$$\Phi(s_{k-1}) = \text{true} \quad \text{and} \quad \Phi(s_k) = \text{false} \tag{2}$$

The rupture point s_k is the *structural failure locus*. The set of all rupture points for invariant Φ constitutes the *structural attack surface induced by the invariant*.

Example 3.1 (Authentication Invariant)**.** Let Φ_{auth} be the invariant: "For every data access operation o, the requesting principal p must be authenticated." Formally:

$$\Phi_{\text{auth}}(s) \equiv \forall o \in \text{Ops}(s) : \text{Auth}(p(o), s) = \text{true} \tag{3}$$

A rupture occurs if $\exists o \in \text{Ops}(s_k) : \text{Auth}(p(o), s_k) = \text{false}$.

3.2 Conceptual Separation

The protocol enforces a strict conceptual separation between four layers of security reasoning:

Table 1: Layered Security Reasoning Framework

Layer	Description	Example
System Properties	CIA + ACDI + Non-repudiation	Confidentiality of user data
Structural Invariants	Rules protecting properties	"Unauthenticated users never access data"
Severity Metrics	Quantified risk scores	CVSS 3.1: 9.8 Critical
Forensic Chain	Cryptographic attestation	SHA-256 + OpenTimestamps

This separation prevents a common epistemic error: conflating the *severity* of a vulnerability (CVSS) with the *structural significance* of an invariant rupture. A low-CVSS configuration error that breaks a fundamental invariant may be more architecturally significant than a high-CVSS buffer overflow in a non-critical component.

3.3 The Co-Architect as Symbiotic Agent

The Security Co-Architect is not an autonomous agent but a *symbiotic cognitive extension*. We define the relationship formally:

Definition 3.3 (Symbiotic Co-Architecture)**.** Let H be the human engineer (Origin Node) and A be the AI agent. The pair (H, A) forms a symbiotic co-architecture if:

(i) H and A share a *joint epistemic state* $\mathcal{E}$ comprising system models, invariants, and hypotheses.

(ii) A may only propose structural modifications after epistemic synchronization with H.

(iii) H retains veto authority over all invariant declarations and rupture confirmations.

(iv) All interactions generate forensic artifacts with SHA-256 attestation.

4 The Symbiotic Architecture Protocol

The Symbiotic Architecture Protocol (SAP) comprises six operational phases, each producing documented, hash-attested outputs. The protocol is iterative: findings in later phases may trigger returns to earlier phases for invariant refinement.

4.1 Phase 0: Conceptual Framework Definition

Objective: Establish the epistemic foundation of the target system.

Inputs: System documentation, architecture diagrams, threat models, regulatory requirements.

Activities:

1. *Property Elicitation:* Identify critical system properties (confidentiality, integrity, availability, atomicity, consistency, isolation, durability, non-repudiation).
2. *Invariant Design:* For each property, derive structural invariants using the predicate formalism of Section 3.1.
3. *Threat Model Integration:* Map known threat actors and attack vectors to potential invariant rupture points.

4. *Standard Compliance:* Identify applicable standards (NIST, ISO 27001, OWASP, GDPR, etc.).

Output: A *Conceptual Framework Document* (CFD) containing:

- Formal invariant definitions with predicate notation.
- Threat model with structural attack surfaces.
- Compliance mapping matrix.
- SHA-256 hash of the document.

4.2 Phase 1: Structural Reconnaissance

Objective: Map the system at the architectural level.

Activities:

1. *Component Mapping:* Identify all architectural components, microservices, databases, queues, and external dependencies.
2. *Data Flow Analysis:* Trace all data flows, classifying them by sensitivity level and trust boundary crossings.
3. *Decision Point Identification:* Locate all authorization, authentication, and business logic decision points.
4. *Dependency Graph Construction:* Build a directed graph $G = (V, E)$ where vertices are components and edges are interfaces.

Output: An *Architectural Map* (AM) comprising:

- Component inventory with trust levels.
- Data flow diagrams with sensitivity annotations.
- Decision point registry.
- Dependency graph G.
- SHA-256 hash of all artifacts.

4.3 Phase 2: Invariant Analysis

Objective: Verify that the invariants defined in Phase 0 hold across the architecture mapped in Phase 1.

Methodology: For each invariant Φ_i, design a *conceptual test*—a logical proof or empirical experiment—to determine whether Φ_i can be violated given the architecture.

Algorithm 1 Invariant Verification Procedure

Require: Invariant set $\{\Phi_1, \ldots, \Phi_k\}$, Architectural Map G
Ensure: Validation report R
$R \leftarrow \emptyset$
for each $\Phi_i \in \{\Phi_1, \ldots, \Phi_k\}$ **do**
  $\text{test}_i \leftarrow \text{DesignConceptualTest}(\Phi_i, G)$
  $\text{result}_i \leftarrow \text{ExecuteTest}(\text{test}_i)$
  if $\text{result}_i = \text{RUPTURE}$ **then**
    $\text{evidence}_i \leftarrow \text{CollectRuptureEvidence}(\Phi_i, G)$
    $R \leftarrow R \cup \{(\Phi_i, \text{RUPTURE}, \text{evidence}_i)\}$
  else
    $R \leftarrow R \cup \{(\Phi_i, \text{PRESERVED}, \emptyset)\}$
  end if
end for
return R

Output: An *Invariant Validation Report* (IVR) containing:

- Per-invariant verdict (PRESERVED / RUPTURE / HYPOTHESIS).
- Empirical or logical evidence for each rupture.
- Structural impact assessment.
- SHA-256 hash of the report.

4.4 Phase 3: System Analysis

Objective: Search for structural failures across seven critical dimensions.

Analysis Dimensions:

1. **Authentication and Session Management:** Invariant preservation across session lifecycle.
2. **Access Control:** Enforcement of authorization invariants at all decision points.
3. **Input Validation:** Structural integrity of trust boundaries against malformed inputs.
4. **Business Logic:** Consistency of workflow invariants (e.g., "a payment cannot be processed without prior inventory reservation").
5. **Cryptography:** Key management invariants, algorithmic soundness, entropy requirements.
6. **Configuration and Hardening:** Invariant preservation under deployment configurations.
7. **Dependencies:** Transitive invariant violations introduced by third-party components.

Failure Classification: Every finding is classified into one of five categories:

Table 2: Structural Failure Taxonomy

Category	Definition
Design Failure	The architecture itself permits invariant violation; no implementation error is required.
Implementation Failure	Correct architecture, but code deviates from the design, causing rupture.
Configuration Failure	Runtime configuration (flags, environment variables) disables or bypasses invariant enforcement.
Process Failure	Organizational or operational procedures (deployment, review, access control) enable rupture.
Dependency Failure	A third-party component introduces an invariant violation not present in the original architecture.

Output: A *Structural Audit Report* (SAR) with classified findings.

4.5 Phase 4: Experimental Confirmation

Objective: Design and document non-destructive experiments for each confirmed structural failure.

Minimum Evidence Requirements:

1. *Exact Reproduction Steps:* A deterministic procedure to trigger the invariant rupture.
2. *Empirical Evidence:* Logs, screenshots, network captures, or code traces demonstrating the rupture.
3. *Invariant Mapping:* Explicit statement of which invariant is violated.
4. *Structural Impact:* Assessment of which system properties degrade as a consequence.

Ethical Safeguards:

- All experiments must be non-destructive.
- No brute-force attacks against production systems.
- Coordinated disclosure compliance (if applicable).
- Chain of custody preservation for all evidence.

Output: A *Reproduction Package* (RP) in English, containing step-by-step PoC documentation.

4.6 Phase 5: Structural Solution Design

Objective: Design architectural remediation that preserves the violated invariant.

Solution Requirements:

1. *Invariant Preservation:* The solution must restore $\Phi(s) = \text{true}$ for all reachable states s.
2. *Risk Neutrality:* The solution must not introduce new invariant violations.
3. *Operational Viability:* The solution must be deployable within the existing operational constraints.
4. *Structural Elegance:* Prefer solutions that reduce architectural complexity rather than adding compensating controls.

Output: A *Structural Solution Design* (SSD) document.

4.7 Phase 6: Strategic Report

Objective: Synthesize all findings into a standardized, legally defensible report.

Report Structure:

1. Executive Summary
2. Severity and Classification (CVSS, CWE, OWASP, Invariant Violated, System Property Impacted)
3. Architectural Context
4. Structural Vulnerability Description
5. Reproduction Steps (Structural Proof of Concept)
6. Impact Analysis
7. Root Cause Analysis
8. Recommended Structural Remediation
9. References
10. Strategic Context

Output: Final *Strategic Security Report* (SSR) with SHA-256 attestation.

5 Forensic Traceability Framework

5.1 The Forensic Chain

Every phase of SAP generates forensic artifacts that form a cryptographically verifiable chain:

1. **SHA-256 Hashing:** Each output document (CFD, AM, IVR, SAR, RP, SSD, SSR) is hashed upon completion. The hash serves as a tamper-evident fingerprint of the document's exact state at the time of attestation.
2. **OpenTimestamps Attestation:** Hashes are submitted to the Bitcoin blockchain via OpenTimestamps [7], creating an immutable temporal proof. This establishes non-repudiable evidence that the document existed in its attested form at a specific point in time.
3. **Chain of Custody:** Each artifact references the hash of its predecessor, forming a Merkle-like chain. The chain links: (a) the invariant definition, (b) the rupture evidence, (c) the remediation proposal, and (d) the attestation timestamp.
4. **Invariant Relationship Mapping:** Every finding explicitly maps to the affected invariant, ensuring traceability from symptom to structural principle.

5.2 Documentation Integrity

The forensic layer measures the *integrity and traceability* of the documentation, not the severity of the security finding. A critical invariant rupture and a minor configuration issue receive the same cryptographic treatment. This decoupling is essential for scientific clarity: conflating documentation quality with vulnerability severity would introduce a category error into the audit process.

The protocol requires that every document entering the forensic chain satisfy minimum structural coherence standards: logical consistency between invariant definitions, rupture evidence, and remediation proposals. Documents failing to meet these standards are returned for revision before attestation.

6 Threat Model and Ethical Assumptions

The Symbiotic Architecture Protocol operates under the following explicit assumptions:

1. **Legitimate Access:** The engineer (Origin Node) possesses legitimate access to the target system, its architecture, and its documentation. SAP is not designed for unauthorized or adversarial reconnaissance.
2. **Architectural Inspectability:** The system architecture is inspectable—components, data flows, and configuration are accessible for analysis. Obfuscated or black-box systems may limit the applicability of invariant-based reasoning.
3. **Non-destructive Operation:** All experimental confirmation (Phase 4) must be conducted in non-destructive environments (staging, testbeds, or isolated sandboxes). Production systems are never the target of active probing without explicit authorization.
4. **Coordinated Disclosure:** Any confirmed invariant rupture with security implications follows a coordinated disclosure process. The protocol includes forensic documentation precisely to support responsible reporting.
5. **Human-in-the-Loop:** The AI agent does not act autonomously on production systems. All invariant declarations, rupture confirmations, and remediation proposals require validation by the Origin Node. The human engineer retains final decision authority over all findings.

These assumptions are not limitations of the protocol per se, but rather ethical and operational preconditions that bound its legitimate application.

7 Case Study: Invariant Rupture in an Authentication Endpoint

To illustrate the operational application of SAP, we present a conceptual case study. The example is simplified for clarity but reflects a pattern commonly encountered in production systems. This case study is presented for pedagogical and methodological demonstration purposes; empirical validation on a live system is left for future work.

7.1 Target System

A web application exposes a REST endpoint `GET /api/v1/users/id/profile` that returns user profile data. The system uses JWT-based session management with an authorization middleware.

7.2 Phase 0: Invariant Definition

The following structural invariant is defined:

$$\Phi_{\text{auth}}(s) \equiv \forall r \in \text{Requests}(s) : \text{Auth}(r) = \text{true} \implies \text{Owner}(r) = \text{Subject}(r) \tag{4}$$

In natural language: "For every request r that accesses user data, the requesting principal must be authenticated, and the requested resource must belong to the authenticated principal unless an explicit authorization grant exists."

7.3 Phase 1: Structural Reconnaissance

The architectural map reveals:

- **Component:** `UserProfileController`
- **Middleware:** `JwtAuthMiddleware` (validates token signature and expiration)
- **Middleware:** `OwnershipCheckMiddleware` (intended to verify resource ownership)
- **Data Flow:** Request → JwtAuthMiddleware → OwnershipCheckMiddleware → Controller → Database

7.4 Phase 2: Invariant Analysis

The conceptual test examines whether Φ_{auth} can be violated. Analysis of the middleware chain reveals that `OwnershipCheckMiddleware` is registered in the development environment but **omitted** from the production deployment configuration due to a deployment script error.

Result: Invariant rupture is possible in production. The predicate Φ_{auth} evaluates to `false` for any request with a valid JWT token targeting another user's `id`.

7.5 Phase 3: Failure Classification

- **Category:** Configuration Failure
- **System Property Impacted:** Confidentiality, Integrity
- **Structural Significance:** High — the invariant is broken at the architectural boundary, not merely in a single endpoint

Note on Severity Decoupling: A traditional CVSS assessment might score this as medium (e.g., 5.3) because it requires a valid token. However, from a structural perspective, the rupture of Φ_{auth} is critical because it invalidates the foundational assumption that authentication implies authorization across the entire API surface.

7.6 Phase 4: Experimental Confirmation

Non-destructive reproduction steps:

1. Register two test accounts: `alice@example.com` and `bob@example.com`.
2. Authenticate as Alice and obtain JWT token T_A.
3. Issue: `GET /api/v1/users/{bob_id}/profile` with header `Authorization: Bearer` T_A.
4. **Expected (secure):** HTTP 403 Forbidden.
5. **Observed (rupture):** HTTP 200 OK with Bob's profile data.

Evidence collected: HTTP request/response logs, JWT payload decoded, middleware chain diff between dev and prod configurations.

7.7 Phase 5: Structural Remediation

The local patch would be: "add `OwnershipCheckMiddleware` to the production middleware stack." The **structural** solution is:

1. **Invariant Enforcement at Build Time:** Introduce a build-time invariant checker that verifies all production routes have the complete middleware chain.

2. **Configuration as Code:** Move middleware registration from environment-specific scripts to a version-controlled, auditable configuration manifest.
3. **Structural Test:** Add an automated invariant test that asserts Φ_{auth} for every authenticated endpoint, not merely for this specific route.

7.8 Phase 6: Forensic Documentation

The reproduction package includes:

- SHA-256 hash of the reproduction script.
- OpenTimestamps attestation of the hash.
- Chain of custody linking the finding to invariant Φ_{auth}.

This case study demonstrates how SAP elevates analysis from "a middleware is missing" to "the authentication-authorization invariant is ruptured at the architectural boundary, and the remediation must preserve the invariant for all future endpoints."

8 Limitations and Scope

The Symbiotic Architecture Protocol has the following explicit limitations:

1. **Not a Replacement for Formal Verification:** SAP does not provide mathematical proofs of correctness. It provides structured reasoning and empirical evidence.
2. **Not an Automated Code Analyzer:** The protocol does not execute code. It reasons about architecture. Static and dynamic analysis tools remain essential complements.
3. **Dependent on Agent Reasoning Capacity:** The quality of invariant analysis depends on the reasoning capabilities of the underlying LLM. Model hallucinations or reasoning failures may produce false negatives or spurious invariant declarations.
4. **Requires Technical Maturity:** The human engineer (Origin Node) must possess sufficient architectural knowledge to validate AI-generated invariants, challenge false hypotheses, and approve rupture confirmations.
5. **Not a Substitute for Penetration Testing:** SAP identifies *where* invariants could rupture; penetration testing demonstrates *how* they can be exploited in practice. Both are necessary.
6. **Scalability Constraints:** For extremely large systems (millions of lines, thousands of microservices), manual epistemic synchronization between human and AI may become a bottleneck. Automated invariant propagation and hierarchical decomposition are subjects of future work.

9 Conclusion

The Master Protocol for Invariant-Based Security Co-Architecture transforms AI-assisted security analysis from a reactive bug hunt into a structured engineering discipline. By focusing on *structural invariants* rather than isolated vulnerabilities, the protocol elevates the conversation from "what is broken?" to "what must never break, and why could it?"

The Symbiotic Architecture Protocol formalizes the relationship between human engineer and AI agent as a *co-architectural partnership* governed by epistemic synchronization, shared model construction, and joint invariant design. This symbiosis is not merely a user interface pattern;

it is a methodological commitment to the idea that security reasoning requires both human judgment and machine scale.

The integration of forensic traceability—SHA-256 hashing and OpenTimestamps attestation—ensures that the protocol's outputs are not merely opinions but *auditable artifacts* with legal and scientific standing.

9.1 Future Work

Several directions warrant further investigation:

1. **Empirical Validation Across Models:** Evaluate SAP performance across different LLM architectures (GPT-4, Claude, Llama, etc.) to assess invariant detection consistency and false positive rates.
2. **Automated Tool Integration:** Develop bridges between SAP and existing security tools (SAST, DAST, SCA) to enable hybrid analysis where machine-discovered vulnerabilities are mapped to architectural invariants.
3. **Domain-Specific Invariant Catalogs:** Build curated libraries of structural invariants for specific domains (financial systems, healthcare, critical infrastructure, blockchain).
4. **Hierarchical Decomposition:** Extend SAP to operate recursively on subsystems, enabling invariant analysis at multiple scales without overwhelming the human engineer.
5. **Legal and Standards Integration:** Formalize the relationship between invariant-based findings and legal frameworks (GDPR Article 32, NIST CSF, ISO 27001 controls) to enable compliance-by-construction.

References

[1] OWASP Foundation. *OWASP Top 10:2021*. 2021. `https://owasp.org/Top10/`

[2] FIRST. *Common Vulnerability Scoring System v3.1: User Guide*. 2019. `https://www.first.org/cvss/v3.1/user-guide`

[3] Hoare, C. A. R. "An Axiomatic Basis for Computer Programming." *Communications of the ACM*, 12(10), 1969.

[4] Meyer, B. *Design by Contract*. Prentice Hall, 1992.

[5] Licklider, J. C. R. "Man-Computer Symbiosis." *IRE Transactions on Human Factors in Electronics*, HFE-1, 1960.

[6] Pearce, H., et al. "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions." *IEEE S&P*, 2022.

[7] OpenTimestamps. *OpenTimestamps: Scalable, Trustless, Distributed Timestamping*. `https://opentimestamps.org/`

[8] Durante, G. E. *SAS Prompt — Symbiotic Agent for Security Co-Architecture*. GitHub, 2026. `https://github.com/Leesintheblindmonk1999/SAS-Prompt-Symbiotic-Agent`